a.

Basic Dataset Statistics:

Training Data	Testing Data	Total
87	29	116

Cougar Face	Cougar Body
69	47

b.

Derivation of MLE formulas:

Let there be a univariate Gaussian data set $y = \{y_1, \dots, y_n\}$

$$p(y_i|\mu, \sigma^2) = N(\mathrm{x}|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$$

Because it is assumed that each observation is independent,

$$p(y|\mu, \sigma^2) = \Pi p(y_i|\mu, \sigma^2) = \Pi \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$$

Therefore, the Negative Log Likelihood function is:

$$NLL(\mu, \sigma^2) = log\ p(y|\mu, \sigma^2) = log\ \Sigma \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x-\mu)^2}$$

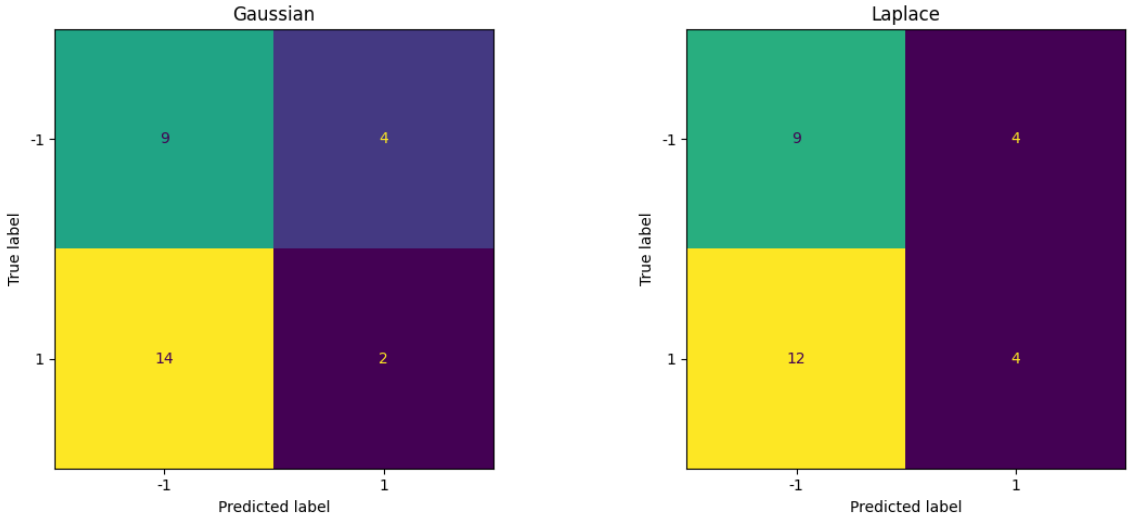
$$= \frac{1}{2\sigma^2}\Sigma(y_n - \mu)^2 + \frac{N}{2} log(2\pi\sigma^2)$$

To minimize the Negative Log Likelihood function, we differentiate by μ and σ^2 and calculate $\frac{d}{d\mu} = 0$ and $\frac{d}{d\sigma^2} = 0$ which gives:

$$\hat{\mu}_{MLE} = \frac{1}{N}\Sigma y_n = \bar{y} \text{ and } \hat{\sigma}^2_{MLE} = \frac{1}{N}\Sigma(y_n - \hat{\mu}_{MLE})^2$$

$$\hat{\sigma}^2_{MLE} = \frac{1}{N}\Sigma y_n^2 - \bar{y}^2$$

Confusion Matrixes:

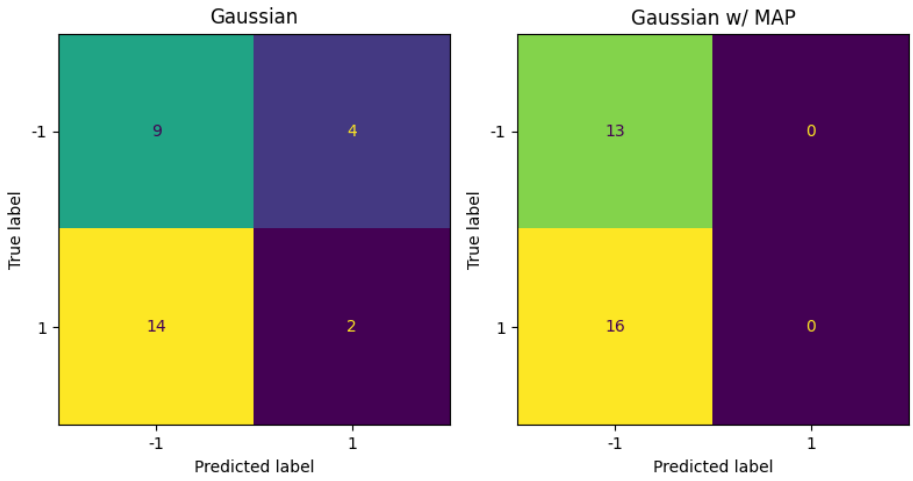


	Gaussian	Laplace
Accuracy	46.96%	47.68%

Both models are generally not accurate, possibly due to the high bias in the dataset itself or caused by overlapping distributions in the features or bad assumptions in feature selection due to the assumption that the data of each pixel is independent of the data of other pixels which both models assume. However, the Laplace model is slightly better for the image dataset as compared to the Gaussian one.

c.

Confusion Matrix:



	Gaussian	Gaussian With MAP
Test Accuracy	46.97%	44.82%

As seen above, the performance of the model with MAP performed significantly worse possibly as the model seems to overly predict the negative class despite input data. Based on my

experimentation through scaling the prior mean $\mu_{0,i}$ and variance ${\sigma^2}_{0,i}$ and also stabilizing my σ^2_{MLE} with a tiny value to ensure it does not become too small, it is most likely due to the specific values in my student ID in combination with the input data and the MAP estimation formula for the Gaussian. As n increases the Denominator of the calculation for σ^2_{MAP} increases causing the value of σ^2_{MAP} to decrease. This means that the variance of the samples is generally low meaning that the samples are clustered around μ which means samples nearer to μ have higher probabilities and vice versa. Therefore, when n is small, due to the smaller denominator and higher σ^2_{MAP} the model becomes more uncertain due to the high variance which prevents overfitting.

Part 2:

a.

By Ordinary Least Squares Criterion:

$Y_i = \theta X_i + \varepsilon_i$ where $\varepsilon_i = N(0, \sigma^2)$ where ε represents noise which follows a Gaussian distribution.

Since $N(0, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(x_i - 0)^2}$ and $\theta_i X_i$ are constant variables, $Y_i = (\theta X_i, \sigma^2)$

Therefore, $Y_i \mid X_i \sim N(\theta X_i, \sigma^2)$ since now given X_i, Y_i now follows a normal distribution centered on θX_i.

That means that $P(Y_i|X_i, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{1}{2\sigma^2}(Y_i - \theta X_i)^2}$

Therefore,

$$NLL(Y_i|X_i, \theta) = -\Sigma \, log\,(P(Y_i|X_i, \theta))$$

$$= -\frac{N}{2} log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\Sigma(Y_i - \theta X_i)^2$$

Where LL stands for log likelihood.

Since we want to maximize the log likelihood,

$$argmax \; -\frac{N}{2} log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\Sigma(Y_i - \theta X_i)^2$$

Since $-\frac{N}{2} log(2\pi\sigma^2)$ and $\frac{1}{2\sigma^2}$ are constants, the formula can be reduced to:

$$argmax \; -\Sigma(Y_i - \theta X_i)^2$$

$$= \; argmin\, \Sigma(Y_i - \theta X_i)^2$$

To optimize this function, we differentiate it against θ to calculate the optimal value of θ.

$$\frac{d}{d\theta}[\Sigma(Y_i - \theta X_i)^2] = 0$$

$$-2\Sigma X_i(Y_i - \theta X_i) \; = \; 0$$

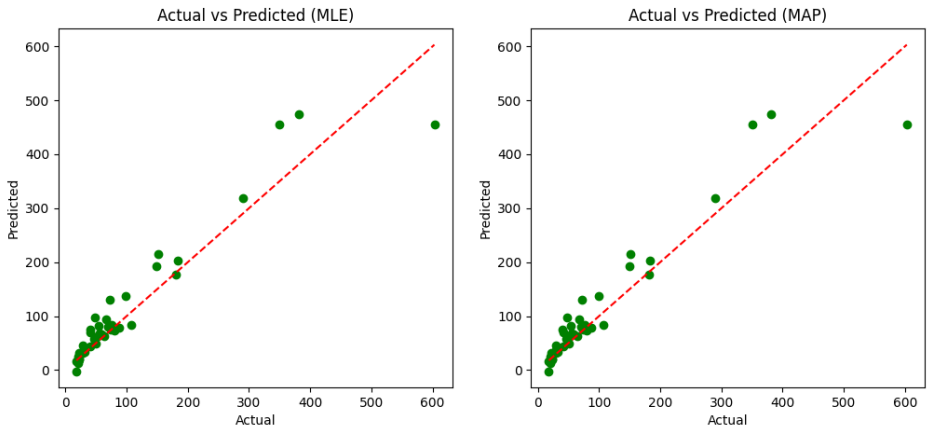
$$\Sigma X_i Y_i = \Sigma X_i X_i^T \theta^T$$

$$X^T Y = X^T X \theta^T$$

$$\theta_{MLE} = (X^T X)^{-1} X^T Y$$

This shows how MLE solution is equivalent to the Ordinary Least Squares Criterion.

Actual vs Predicted Graphs for both models

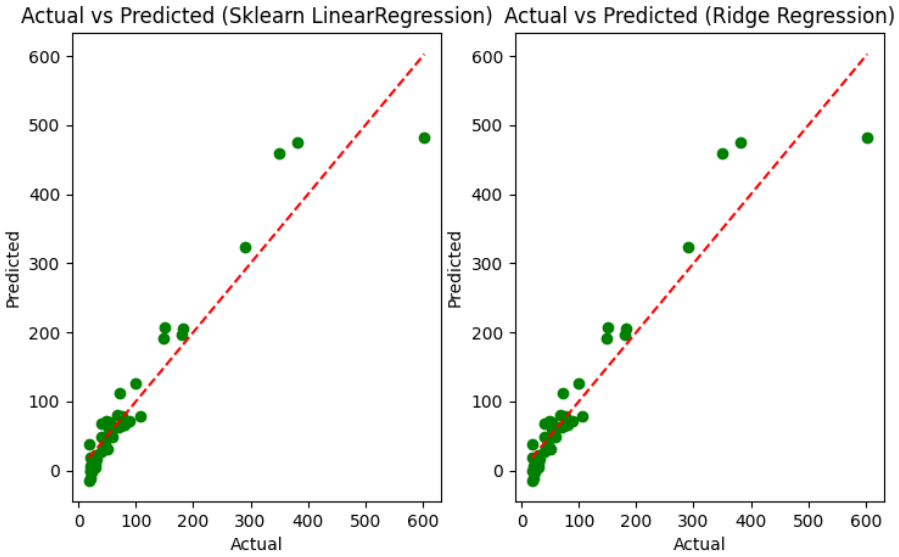


	MLE	MAP
Mean Square Error	1472.91	1472.90

The addition of λI improves the stability of the matrix inversion of $(X^T X)^{-1}$ as the matrix is not guaranteed to be invertible if there are more features than samples (unlikely) or $determinant = 0$ which guarantees the matrix would not be invertible. The matrix may also be nearly singular if features are highly correlated. The addition of λI here would guarantee invertibility as even if the determinant of the original matrix is 0, the new determinant of $(X^T X + \lambda I)$ is guaranteed to be λ. It also helps to reduce the condition number, which measures the stability through this equation $K = \frac{largest\ eigenvalue}{smallest\ eigenvalue}$ by increasing the smallest eigenvalue proportionally more than the largest eigenvalue, causing K to decrease. Lastly, the addition also helps to regularize the model by pulling θ towards 0 depending on the value of λ.

b.

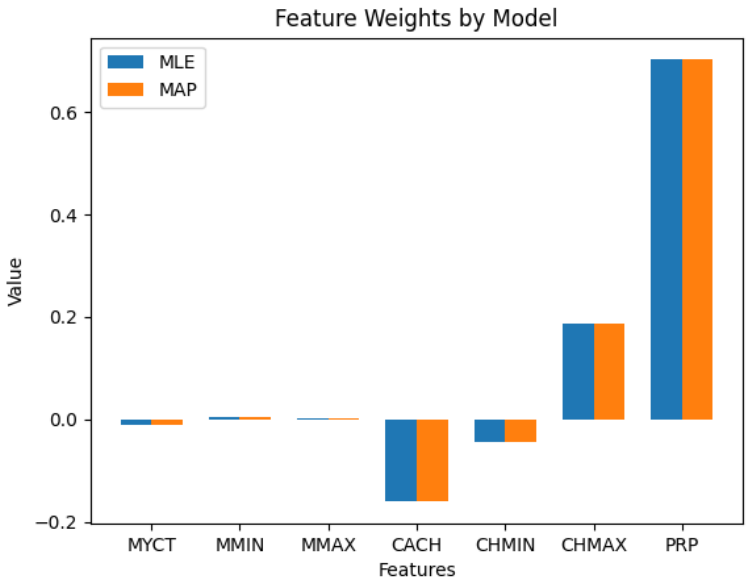
Actual vs Predicted Graphs for the S:



	Scratch Linear Regression (MLE)	Scratch Ridge Regression (MAP)	Sklearn Linear Regression	Sklearn Ridge Regression
Mean Squared Error	1472.91	1472.90	1308.39	1308.39

The discrepancy between my models and the Scikit-Learn results are most likely caused by the higher performance algorithms implemented in the Scikit-Learn models since they are written to be generally more reliable, optimized and robust due to their more tested and debugged code as compared to from scratch.

Feature Weights by Model:



Even though it is not very noticeable on the plot, the weights of the MAP model are slightly smaller than the MLE models as seen in the table below:

MLE Weights	MAP Weights
-0.01049697	0.70213445
0.00440107	0.00440106
0.00204118	0.00204118
-0.16107224	-0.16107193
-0.04526019	-0.04525805
0.18604451	0.18604386
0.70213445	0.70213433

The lack of noticeable effect of the Gaussian Prior is most likely due to the sum of my student ID divided by 100 being relatively small (0.20) compared to the input features which range between 0 – 32000. The high input feature values cause the weights (as seen above) to be generally small, causing the penalty term in the loss function $\lambda|\theta|^2$ to stay moderate meaning the regularizing effect of the MAP to be smaller. As mentioned previously, the MAP weights are generally smaller than the MLE weights as the Loss function for the model can be represented:

$$Loss = DataLoss + \lambda\left|\left|\theta\right|\right|^2$$

Data loss depends on the scale of the features, meaning it is constant if features are constistenly scaled and the λ term penalizes larger weights depending on how small lambda is. The smaller λ is the more the larger weights are penalized and pulled towards 0, resulting in smaller magnitude weights causing the shrinkage as described. This achieves regularization as it restricts model complexity by ensuring previously large weights are less able to influence small changes to those features by noise. Therefore, the model becomes simpler as the model becomes less reliant on any single feature reducing overfitting.

Generally speaking, the MAP model would be more likely to generalize unseen data better due to the higher possibility of overfitting in the MLE model causing it to fit to noise which is likely in most datasets. From a Bias-Variance Tradeoff perspective, MLE has lower bias but higher variance while MAP has a slightly higher bias and a lower variance. The MLE has a near 0 bias due to it fitting the training data very closely due the lack of regularization on the model but that causes the variance to generally be higher due to the model tending to fit to noise present in its training data. In comparison, MAP has a slightly higher bias due to the prior assumption that is used as a regularization term not fully aligning with reality but the lower variance due to the reduced sensitivity to training noise allows the MAP model to perform better for unseen hardware data.

Main.py for Part 1:

```
import cv2
import os
import matplotlib.pyplot as plt
import numpy as np
import numpy.typing as npt
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import train_test_split

from Predictors import GaussSimple, LaplacePredictor, GaussWithMap

np.random.seed(0)



def preprocess_image(path, size=(64, 64)):
    img = cv2.imread(path)
    if img is None:
        return
    img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    img = cv2.resize(img, size)
    img = img.astype("float32") / 255.0
    return img.flatten()

def build_data(dataset_dir):
    labels = []
    rows = []

    for label, folder in [(1, "cougar_face"), (-1, "cougar_body")]:
        folder_path = os.path.join(dataset_dir, folder)

        for fname in os.listdir(folder_path):
            if not fname.lower().endswith(".jpg"):
                continue
            path = os.path.join(folder_path, fname)
            row = np.hstack([
                label
            ])
            row2 = preprocess_image(path)
            labels.append(row)
            rows.append(row2)
    labels = np.array(labels)
    rows = np.array(rows)
    return rows, labels

def calcMuAndSigma(S: npt.NDArray) -> tuple[npt.NDArray, npt.NDArray]:
    mean = S.mean() / 7
    mu0 = np.full(4096, mean * 255)
```

```
    sigma0 = np.arange(4096)
    sigma0 = (S.var() * (1 + (sigma0 % S.max()) / 10))
    # print(f"mu0: {mu0}")
    # print(f"sigma0: {sigma0}")
    return sigma0, mu0


def PlotConfusionMatrix(
        Xtrain : npt.ArrayLike, ytrain : npt.ArrayLike,
        Xtest : npt.ArrayLike, ytest : npt.ArrayLike,
        mu0: npt.ArrayLike, sigma0: npt.ArrayLike) -> None:
    classifiers = {
        "Gaussian": GaussSimple(),
        "Gaussian w/ MAP": GaussWithMap(mu0, sigma0),
        "Laplace": LaplacePredictor()
    }
    fig, axes = plt.subplots(1, 3, figsize=(12, 5))

    for i, (key, classifier) in enumerate(classifiers.items()):
        classifier.fit(Xtrain, ytrain)
        yPred = classifier.predict(Xtest)
        print("accuracy: ", np.mean(yPred==ytest))
        ConfusionMatrixDisplay.from_predictions(ytest, yPred, ax=axes[i],
colorbar=False)
        axes[i].set_title(key)


    plt.tight_layout()
    plt.show()


def __main__():
    S = np.array([0, 2, 7, 3, 1, 2, 5])
    sigma0, mu0 = calcMuAndSigma(S)

    X, y = build_data("caltech_cougar")
    Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, test_size=0.25,
shuffle=True)
    print(Xtrain)
    ytrain = ytrain.ravel()

    PlotConfusionMatrix(Xtrain, ytrain, Xtest, ytest, mu0, sigma0)



__main__()
```

Predictors.py for Part 1:

```python
import numpy as np

import numpy.typing as npt

class GaussSimple:
    def __init__(self):
        self.priors = np.array([])
        self.classes = np.array([])
        self.mu = np.array([])
        self.sigma2 = np.array([])

    def fit(self, Xtrain, ytrain):
        self.classes = np.unique(ytrain)
        nClasses = len(self.classes)
        nFeatures = Xtrain.shape[1]

        self.mu = np.zeros((nClasses, nFeatures))
        self.sigma2 = np.zeros((nClasses, nFeatures))
        self.priors = np.zeros(nClasses)

        for i, c in enumerate(self.classes):
            X_c = Xtrain[ytrain == c]
            self.mu[i] = np.mean(X_c, axis=0)
            self.sigma2[i] = np.var(X_c, axis=0)
            self.priors[i] = len(X_c) / len(ytrain)

    def predict(self, Xtest):
        nSamples = Xtest.shape[0]
        nClasses = len(self.classes)

        logPosteriors = np.zeros((nSamples, nClasses))

        for i in range(nClasses):
            logPriors = np.log(self.priors[i])
            logProbsP1 = -0.5 * np.log(2 * np.pi * self.sigma2[i])
            logProbsP2 = -0.5 * ((Xtest - self.mu[i]) ** 2) / self.sigma2[i]
            logProbs = logProbsP1 + logProbsP2
            logLikelihood = np.sum(logProbs, axis=1)
            logPosteriors[:, i] = logPriors + logLikelihood

        predictions = self.classes[np.argmax(logPosteriors, axis=1)]

        return predictions

class LaplacePredictor:
    def __init__(self):
        self.priors = np.array([])
        self.classes = np.array([])
```

```
        self.mu = np.array([])
        self.b = np.array([])

    def fit(self, Xtrain, ytrain):
        self.classes = np.unique(ytrain)
        nClasses = len(self.classes)
        nFeatures = Xtrain.shape[1]

        self.mu = np.zeros((nClasses, nFeatures))
        self.b = np.zeros((nClasses, nFeatures))
        self.priors = np.zeros(nClasses)

        for i, c in enumerate(self.classes):
            X_c = Xtrain[ytrain == c]
            self.mu[i] = np.median(X_c, axis=0)
            self.b[i] = np.mean(np.abs(X_c - self.mu[i]), axis=0)
            self.priors[i] = len(X_c) / len(ytrain)

    def predict(self, Xtest):
        nSamples = Xtest.shape[0]
        nClasses = len(self.classes)

        logPosteriors = np.zeros((nSamples, nClasses))

        for i in range(nClasses):
            logPriors = np.log(self.priors[i])
            logProbsP1 = -np.log(2 * self.b[i])
            logProbsP2 = -np.abs(Xtest - self.mu[i]) / self.b[i]
            logProbs = logProbsP1 + logProbsP2
            logLikelihood = np.sum(logProbs, axis=1)
            logPosteriors[:, i] = logPriors + logLikelihood

        predictions = self.classes[np.argmax(logPosteriors, axis=1)]

        return predictions


class GaussWithMap(GaussSimple):
    def __init__(self, mu0, sigma0):
        super().__init__()
        self.mu0 = mu0
        self.sigma0 = sigma0

    def fit(self, Xtrain, ytrain):
        self.classes = np.unique(ytrain)
        nClasses = len(self.classes)
        nFeatures = Xtrain.shape[1]

        self.mu = np.zeros((nClasses, nFeatures))
```

```
        self.sigma2 = np.zeros((nClasses, nFeatures))
        self.priors = np.zeros(nClasses)
        var = np.var(Xtrain, axis=0)
        for i, c in enumerate(self.classes):
            X_c = Xtrain[ytrain == c]
            mean = np.mean(X_c, axis=0)
            # var = np.var(X_c, axis=0)
            n = len(X_c)
            self.mu[i] = (var * self.mu0 + n * self.sigma0 * mean) / (var + n
* self.sigma0)
            self.sigma2[i] = (var * self.sigma0) / (var + n * self.sigma0)
            self.priors[i] = n / len(ytrain)
        with open("fit.txt", "w") as text_file:
            text_file.write("\n=== MODEL PARAMETERS ===\n")
            for i, c in enumerate(self.classes):
                text_file.write(f"\nClass {c}:\n")
                text_file.write(f"  Prior: {self.priors[i]:.6f}\n")
                text_file.write(f"  log(prior):
{np.log(self.priors[i]):.6f}\n")
                text_file.write(f"  mu mean: {self.mu[i].mean():.6f}\n")
                text_file.write(f"  mu std: {self.mu[i].std():.6f}\n")
                text_file.write(f"  sigma2 mean:
{self.sigma2[i].mean():.10f}\n")
                text_file.write(f"  sigma2 min:
{self.sigma2[i].min():.10f}\n")
                text_file.write(f"  sigma2 max:
{self.sigma2[i].max():.10f}\n")

            text_file.write("\n=== PRIOR PARAMETERS ===\n")
            text_file.write(f"mu0 mean: {self.mu0.mean():.6f}\n")
            text_file.write(f"mu0 std: {self.mu0.std():.6f}\n")
            text_file.write(f"sigma0 mean: {self.sigma0.mean():.10f}\n")
            text_file.write(f"sigma0 min: {self.sigma0.min():.10f}\n")
            text_file.write(f"sigma0 max: {self.sigma0.max():.10f}\n")

    def predict(self, Xtest):
        nSamples = Xtest.shape[0]
        nClasses = len(self.classes)

        logPosteriors = np.zeros((nSamples, nClasses))

        eps = 1e-6 # eps is for stability to prevent any index of sigma2 from
being 0

        with open("test.txt", "w") as text_file:
            text_file.write("\n=== FIRST TEST SAMPLE BREAKDOWN ===\n")
            for i in range(nClasses):
                logPriors = np.log(self.priors[i])
```

```
                sigma2_stable = self.sigma2[i] + eps

                logProbsP1 = -0.5 * np.log(2 * np.pi * sigma2_stable)
                logProbsP2 = -0.5 * ((Xtest - self.mu[i]) ** 2) /
sigma2_stable
                logProbs = logProbsP1 + logProbsP2
                logLikelihood = np.sum(logProbs, axis=1)
                logPosteriors[:, i] = logPriors + logLikelihood
                text_file.write(f"\nClass {self.classes[i]}:\n")
                text_file.write(f"  log prior: {logPriors:.6f}\n")
                text_file.write(f"  sum(log normalization):
{logProbsP1.sum():.6f}\n")
                text_file.write(f"  sum(log distance penalty):
{logProbsP2.sum():.6f}\n")
                text_file.write(f"  total log likelihood: {(logProbsP1.sum() +
logProbsP2.sum()):.6f}\n")
                text_file.write(f"  log posterior: {logPosteriors[0,
i]:.6f}\n")

        predictions = self.classes[np.argmax(logPosteriors, axis=1)]

        return predictions
```

Main.py for Part 2:

```
import os
import matplotlib.pyplot as plt
import numpy as np
import numpy.typing as npt
import pandas as pd
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

from Predictors import ScratchLinReg, LinearRegressionMAP

def buildData() -> tuple[npt.ArrayLike, npt.ArrayLike]:
    filePath = os.path.join('computer+hardware', 'machine.data')
    df = pd.read_csv(filePath, header=None, names=[
        'vendor', 'model', 'MYCT', 'MMIN', 'MMAX',
        'CACH', 'CHMIN', 'CHMAX', 'PRP', 'ERP'
    ])

    print("First few rows:")
    print(df.head())
    print("\nData types:")
    print(df.dtypes)
    print("\nBasic statistics:")
    print(df.describe())

    X = df[['MYCT', 'MMIN', 'MMAX','CACH', 'CHMIN', 'CHMAX', 'PRP']].values
    Y = np.array(df['ERP'].values)
    Y = Y.reshape(-1, 1)

    print(f"\nX shape: {X.shape}")
    print(f"Y shape: {Y.shape}")

    return X, Y


def plotActualVsPredicted(Xtrain: npt.NDArray, ytrain: npt.NDArray, Xtest:
npt.NDArray, ytest: npt.NDArray):
    models = {
        "MLE" : ScratchLinReg(),
        "MAP" : LinearRegressionMAP(),
        "Sklearn LinearRegression": LinearRegression(),
        "Ridge Regression": Ridge()
    }

    fig, axes = plt.subplots(1, len(models), figsize=(18, 5))

    for i, (key, model) in enumerate(models.items()):
```

```
        model.fit(Xtrain, ytrain)
        yPred = model.predict(Xtest)
        mse = mean_squared_error(ytest, yPred)
        print(f"MSE of {key}: {mse}")
        axes[i].scatter(ytest, yPred, color='green', label="Actual Data")
        axes[i].set_title(f"Actual vs Predicted ({key})")
        axes[i].set_xlabel('Actual')
        axes[i].set_ylabel('Predicted')
        axes[i].plot([min(ytest), max(ytest)], [min(ytest), max(ytest)],
color='red', linestyle='--')
    plt.show()

def plotWeights(Xtrain: npt.NDArray, ytrain: npt.NDArray, Xtest: npt.NDArray,
ytest: npt.NDArray):
    models = {
        "MLE" : ScratchLinReg(),
        "MAP" : LinearRegressionMAP(),
    }

    features = ['MYCT', 'MMIN', 'MMAX','CACH', 'CHMIN', 'CHMAX', 'PRP']

    x = np.arange(len(features))
    width = 0.35
    models['MLE'].fit(Xtrain, ytrain)
    models['MAP'].fit(Xtrain, ytrain)
    plt.bar(x - width/2, models['MLE'].theta.ravel(), width=width,
label=f"MLE")
    plt.bar(x + width/2, models['MAP'].theta.ravel(), width=width,
label=f"MAP")
    print(models['MLE'].theta.ravel())
    print(models['MAP'].theta.ravel())
    plt.xticks(x, features)
    plt.legend()
    plt.title("Feature Weights by Model")
    plt.xlabel('Features')
    plt.ylabel('Value')
    plt.show()


def __main__():
    X, Y = buildData()
    Xtrain, Xtest, ytrain, ytest = train_test_split(X, Y, test_size=0.2,
random_state=0, shuffle=True)
    plotWeights(Xtrain, ytrain, Xtest, ytest)
```

```
__main__()
```

Predictors.py for Part 2:

```
import numpy as np
import numpy.linalg as npl
import numpy.typing as npt


class ScratchLinReg:
    def __init__(self):
        self.theta = np.array([])

    def fit(self, Xtrain : npt.NDArray, ytrain : npt.NDArray):
        XtX = Xtrain.T @ Xtrain
        XtXInv = npl.inv(XtX)
        XtY = Xtrain.T @ ytrain
        self.theta = XtXInv @ XtY


    def predict(self, Xtest):
        return Xtest @ self.theta

class LinearRegressionMAP(ScratchLinReg):
    def __init__(self):
        super().__init__()
        self.l = (0 + 2 + 7 + 3 + 1 + 2 + 5) / 100.0 # Lambda Calculation

    def fit(self, Xtrain : npt.NDArray, ytrain : npt.NDArray):
        XtX = Xtrain.T @ Xtrain
        XtXInv = npl.inv(XtX + self.l * np.identity(XtX.shape[0]))
        XtY = Xtrain.T @ ytrain
        self.theta = XtXInv @ XtY
```

AI Declaration:

AI was used here for general bug fixing for code as well as research on the mathematical derivations for how the MLE solution is equivalent to the Ordinary Least Squares as well for understanding the effects of λ in ridge regression.